

Exactly. We should ****not install OCR yet****. First we'll understand and implement ****Stage 1.4.2.2 – Detecting Scanned / Non-Text PDF Pages****.

The goal is to build a small inspection experiment in your existing notebook:

```

`text
notebooks/
    text-loaders/
        01_data_ingestion.ipynb
`

```

1. What are we trying to detect?

For every PDF page, we want to answer:

```
```text
Does this page contain useful extractable text?
 ■
 ■■■■■■■■■■
 ■ ■
 YES NO
 ▼ ▼
Normal PDF OCR candidate
```
```

But we'll go one step further.

A page can contain **some text** and still be problematic. So we'll collect useful diagnostics rather than simply returning ``True`/`False``.

— — —

1.4.2.2 – PDF Page Inspection

Step 1 – Create the inspection function

Add a new Markdown cell:

```
```markdown
1.4.2.2 Detecting Scanned / Non-Text PDF Pages
```

The objective is to inspect every PDF page before deciding whether normal text extraction is sufficient or OCR may be required.

Then add this code cell:

```

"""python
def inspect_pdf_documents(name, documents):
 """
 Inspect LangChain Documents produced from a PDF.

 Reports:
 - page number
 - extracted character count
 - whether text exists

```

```

- source metadata
"""

print("=" * 80)
print(f"PDF INSPECTION: {name}")
print("=" * 80)

print(f"Number of pages/documents: {len(documents)}")

for index, document in enumerate(documents, start=1):

 content = document.page_content.strip()

 print(f"\nPage {index}")
 print("-" * 80)

 print("Characters :", len(content))
 print("Has text :", bool(content))
 print("Metadata :", document.metadata)
...

```

---

# Step 2 – Inspect the text-based PDF

Run:

```

```python
inspect_pdf_documents(
    "Text-based PDF",
    text_documents
)
```

```

You should see something conceptually similar to:

```

```text
=====
PDF INSPECTION: Text-based PDF
=====

Number of pages/documents: 5

Page 1
-----
Characters : *****
Has text   : True

Page 2
-----
Characters : *****
Has text   : True
```

```

and so on.

The exact character counts aren't important.

The important observation is:

```
```text
Has text : True
```
```

for the pages containing extractable text.

---

# Step 3 – Inspect the scanned PDF

Now run:

```
```python
inspect_pdf_documents(
    "Scanned PDF",
    scanned_documents
)
```
```

You should see something similar to:

```
```text
=====
PDF INSPECTION: Scanned PDF
=====

Number of pages/documents: 1

Page 1
-----
Characters : 0
Has text    : False
```
```

This is exactly what we want to demonstrate.

We have now programmatically detected:

```
```text
Text PDF

Page 1 → text available
Page 2 → text available
```
```

versus:

```
```text
Scanned PDF

Page 1 → no extractable text
```
```

---

# Step 4 – Make the detection reusable

The previous function is useful for learning, but let's improve it.

Create:

```
```python
def analyze_pdf_pages(documents):
    """
    Analyze PDF pages and return page-level diagnostics.
    """

    results = []

    for page_number, document in enumerate(documents, start=1):

        content = document.page_content.strip()

        results.append({
            "page": page_number,
            "characters": len(content),
            "has_text": bool(content),
            "metadata": document.metadata
        })

    return results
```
```

Now execute:

```
```python
text_analysis = analyze_pdf_pages(text_documents)

scanned_analysis = analyze_pdf_pages(scanned_documents)
```
```

Inspect:

```
```python
text_analysis
```
```

and:

```
```python
scanned_analysis
```
```

You'll get Python dictionaries representing each page.

---

# Step 5 – Make the result easier to understand

Because you're using a notebook, let's use a DataFrame.

```
```python
import pandas as pd
```
```

Then:

```
```python
text_df = pd.DataFrame(text_analysis)

text_df
```
```

You should get something conceptually like:

| page | characters | has_text |
|------|------------|----------|
| 1    | 850        | True     |
| 2    | 720        | True     |
| 3    | 940        | True     |
| 4    | 650        | True     |
| 5    | 810        | True     |

Now:

```
```python
scanned_df = pd.DataFrame(scanned_analysis)

scanned_df
```
```

You should see:

| page | characters | has_text |
|------|------------|----------|
| 1    | 0          | False    |

This makes the difference very obvious.

---

# Step 6 – Don't stop at `has\_text`

This is an important production-level concept.

Consider this PDF page:

```
```text
Page 3

Characters: 37
Has text: True
```
```

Does that automatically mean the page is fine?

**\*\*No.\*\***

```
```text
```



- * a large table
- * an image containing text
- * a scanned signature
- * a diagram
- * a two-column layout
- * text extracted in the wrong order

```
> **Text presence is a detection signal, not an extraction-quality guarantee.**
```

— — —

Let's create a simple learning-oriented classifier.

```
```python
```

```
def classify_page(content, minimum_characters=50):
```

11 11 11

Simple learning-oriented classification.

Returns:

TEXT\_AVAILABLE

POSSIBLE\_SCANNED

■ ■ ■

```
character_count = len(content.strip())
```

```
if character_count >= minimum_characters:
```

```
return "TEXT_AVAILABLE"
```

```
return "POSSIBLE_SCANNED"
```

...

```
```python
```

```
for document in text_documents:
```

```

        result = classify_page(document.page_content)
    print(result)
...

```

And:

```

```python
for document in scanned_documents:

 result = classify_page(document.page_content)

 print(result)
...

```

Your scanned page should be classified as:

```

```text
POSSIBLE_SCANNED
```

```

---

# Step 8 – Why do we call it `POSSIBLE\_SCANNED`?

This naming is intentional.

We should **not** say:

```

```text
No text → definitely scanned PDF
```

```

because there are other possibilities.

For example:

```

```text
No extracted text
    ■
    ■■■ Scanned page
    ■
    ■■■ Image-only page
    ■
    ■■■ Extraction failure
    ■
    ■■■ Unsupported encoding
    ■
    ■■■ Corrupted/unusual PDF
...

```

Therefore our ingestion pipeline should say:

```

```text
POSSIBLE_SCANNED
```

```

rather than:

```
```text
DEFINITELY_SCANNED
```
```

This is a much better engineering mindset.

Step 9 – Build our first PDF inspection report

Let's combine everything:

```
```python
def generate_pdf_inspection_report(documents, minimum_characters=50):

 report = []

 for page_number, document in enumerate(documents, start=1):

 content = document.page_content.strip()
 character_count = len(content)

 if character_count >= minimum_characters:
 status = "TEXT_AVAILABLE"
 else:
 status = "POSSIBLE_SCANNED"

 report.append({
 "page": page_number,
 "characters": character_count,
 "status": status,
 "source": document.metadata.get("source"),
 })

 return report
```
```

Run:

```
```python
report = generate_pdf_inspection_report(
 scanned_documents
)

pd.DataFrame(report)
```
```

You should get something similar to:

| page | characters | status | source |
|------|------------|------------------|--------|
| 1 | 0 | POSSIBLE_SCANNED | ... |

Now test the normal PDF:

Step 11 – One important improvement

```
```python
document.page_content
```
```

For our next experiment we'll inspect the PDF itself for:

```

` ``text
Page
■
■■■ Text blocks
■■■ Images
■■■ Image count
■■■ Text character count
■■■ Text density
■■■ Potentially suspicious pages
` ``

```

```

    ``text
    PDF PAGE
    [Diagram showing a flow from PDF PAGE to Text, Images, and Layout, then to Page Analysis, and finally to Ingestion Decision.]
    ````

```

### Your learning progression is now:

```

 ``text
1.4.2 OCR & Complex PDF Ingestion
 ■
 ■■■ 1.4.2.1 Text PDF vs Scanned PDF ✓
 ■
 ■■■ 1.4.2.2 Detect Non-Text Pages ← We are here
 ■
 ■■■ 1.4.2.3 Deep PDF Page Analysis
 ■
 ■■■ 1.4.2.4 OCR Fundamentals
 ■
 ■■■ 1.4.2.5 OCR Implementation
 ■
 ■■■ 1.4.2.6 OCR → LangChain Documents
 ``

```

**\*\*Don't install an OCR engine yet.\*\*** First complete the page-analysis experiment; it will make the reason for OCR much clearer and will give you a more production-oriented mental model of document ingestion.